# REPORT GENAI-EXPERIMENT 3

**Name:** Raj Mathuria **Batch:** C

**UID:** 2023300139

**Link:** https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp3

## Objective

To implement the Scaled Dot-Product Attention mechanism with Python and NumPy, calculate attention scores and weights from Query (Q), Key (K), and Value (V) matrices, obtain the resulting attention output, and visualize the attention distribution through a heatmap.

## Theory

### Attention

Attention enables a model to emphasize the most relevant elements of an input sequence. It assigns different weights to tokens according to their importance and uses Query, Key, and Value matrices to determine how strongly each token should be attended to.

### Self-Attention

Self-attention is an attention mechanism in which Q, K, and V originate from the same input sequence. This allows every token to interact with the others and capture relationships between tokens, including relationships between distant positions.

## Algorithm

1. Use the Q, K, and V matrices as the inputs.
2. Compute the attention scores using $S = QK^T$.
3. Scale the scores by $\sqrt{d_k}$.
4. Apply Softmax to convert the scaled scores into attention weights.
5. Multiply the attention weights by V.
6. Use the result as the final attention output.

## Matrix Dimensions and Experimental Modifications

| Stage | Matrix Operation | Expected Dimension |
|---|---|---|
| Input | Q, K, V | 3 × 2 |
| Similarity | QK■ | 3 × 3 |
| Scaling | QK■ / √d■ | 3 × 3 |
| Normalization | softmax(scores) | 3 × 3 |
| Aggregation | Weights × V | 3 × 2 |

## Experimental Modifications

### Changing the Q matrix

Changing the Query matrix directly modifies the way each query token attends to the key tokens. In the reported experiment, the first attention-weight row changed substantially, from [0.401, 0.198, 0.401] to [0.198, 0.401, 0.401]. This indicates stronger attention toward the second key token. The second and third rows remained unchanged because their corresponding query rows were similar to those in the original matrix.

### Changing K while keeping Q and V fixed

Altering the Key matrix changes the attention weights because the weights depend on the Q–K dot products. In the reported case, the first query token attended almost equally to all three keys, with weights [0.333, 0.333, 0.333]. The other two rows remained unchanged because their similarity relationships with the modified key entries were preserved.

### Changing V while checking the attention weights

Changing V does not alter the attention weights because those weights are determined by Q and K. It does, however, change the final attention output because the output is a weighted combination of V. Scaling V by a factor of 10 produced an output approximately 10 times larger than the original output.

# Effect of Scaling

## Comparing attention with and without √d■ scaling

Dividing the attention scores by √d■ reduces their magnitude before Softmax is applied. Without this scaling, large dot products can make Softmax excessively sharp and can produce very small gradients. The reported comparison shows that scaling creates a smoother attention distribution and reduces the tendency for a single key token to dominate.

## Increasing the embedding dimension

As d■ becomes larger, the magnitude of raw dot-product scores tends to increase. Without scaling, Softmax therefore becomes sharper, causing a small number of tokens to receive most of the attention. For d■ = 100, the unscaled attention was reported to be highly concentrated, approaching one-hot behavior, which can make learning diverse relationships more difficult.

With √d■ scaling, the d■ = 100 attention distribution was substantially more spread out. Scaling therefore helps preserve a more balanced relationship between query and key tokens.

## Increasing the number of tokens

When the sequence grows from three tokens to four or five, the attention assigned by each query is distributed across more key tokens. As each row of attention weights must sum to one, the individual weights generally become smaller as the number of tokens increases.

# Implementing Self-Attention

The self-attention implementation processes an input X using randomly initialized WQ, WK, and WV matrices to obtain Q, K, and V. The resulting attention weights are displayed in the heatmap and show how strongly each token relates to the others.

In the reported output, the second token received the greatest attention from every query token. For example, Query Token 1 assigned a weight of 0.454 to Key Token 2. This indicates a strong contextual relationship with the second token for the current projections, which then contributes to the context-aware output.

## Self-Attention Visualization

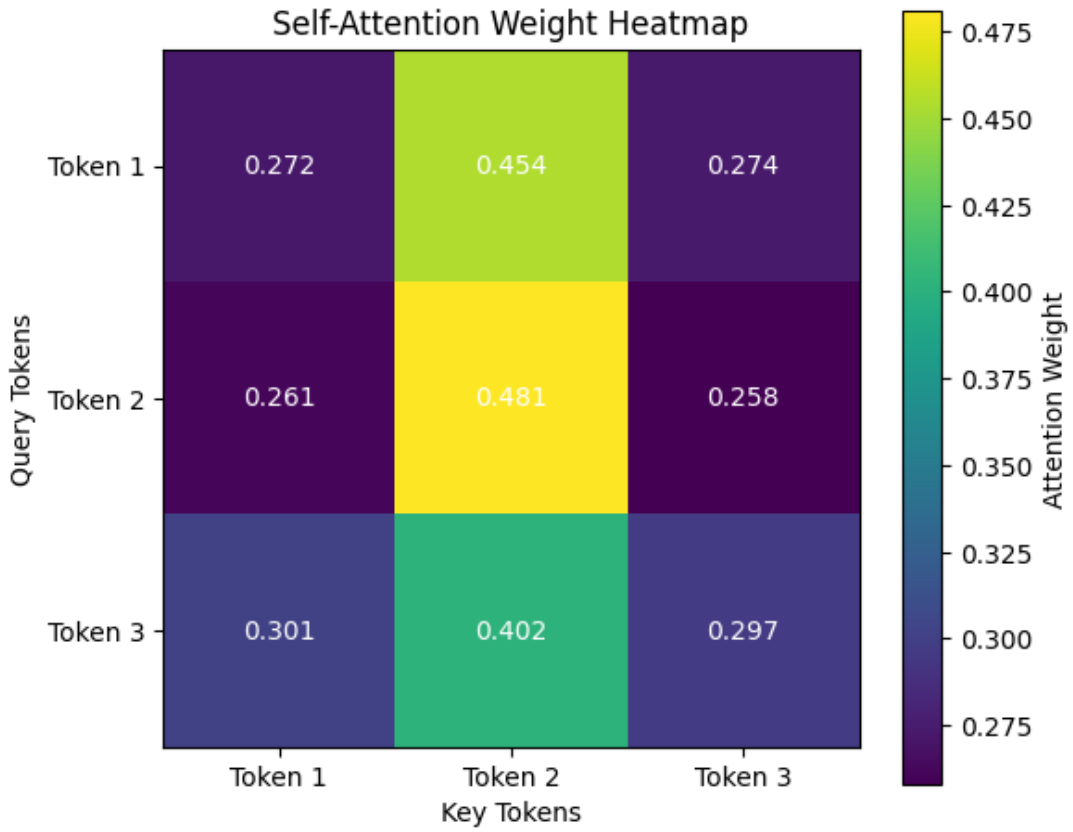



Figure 1. Self-Attention Weight Heatmap from the original experiment.

## Reading the Heatmap

The rows represent query tokens and the columns represent key tokens. Each cell shows the attention weight assigned by a query token to a key token. The highest reported value is 0.481 for Query Token 2 attending to Key Token 2, while Query Token 1 assigns 0.454 to Key Token 2 and Query Token 3 assigns 0.402 to Key Token 2. The visualization therefore supports the observation that the second token receives the strongest attention across the three query positions.

## Conclusion

The experiments demonstrate that the Query and Key matrices primarily determine the attention weights and therefore control how tokens relate to one another contextually. The Value matrix changes the content of the resulting attention output without changing the weights. Scaling by √d■ is important for preventing overly concentrated attention distributions, particularly when the embedding dimension becomes large. As the sequence length grows, attention is distributed across more tokens. The experiment also illustrates how semantic representations such as GloVe can support meaningful relationships between words within an attention mechanism.

## Repository

**Experiment 3 code:**
https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp3

The repository link above replaces the Colab Notebook reference used in the source report.